

Register Number: 192425317

Name: M. Sahasra

CO2 – AT2

MLA0201 – Fundamentals of Machine Learning

Scenario-Based

Question 1:

Scenario: Apply a linear regression model to predict apartment rent based on floor area, number of rooms, and location in a real-estate scenario.

Answer:

Introduction:

Linear Regression is a supervised machine learning algorithm used to predict continuous values. In this scenario, it is used to predict the rent of an apartment based on features such as floor area, number of rooms, and location.

Given Data (Sample):

Floor Area (sq.ft)	Rooms	Location	Rent (₹)
800	2	City	18000
1000	3	City	25000
1200	3	Suburb	22000
1500	4	City	32000
900	2	Suburb	19000

Solution:

Step 1: Collect Data

The apartment details such as:

- Floor Area
- Number of Rooms
- Location
- Rent

are collected.

Step 2: Train the Model

The Linear Regression algorithm learns the relationship between apartment features and rent.

General Equation:

$$\text{Rent} = \beta_0 + \beta_1(\text{Floor Area}) + \beta_2(\text{Rooms}) + \beta_3(\text{Location})$$

where,

- β_0 = Intercept
- β_1 = Area coefficient
- β_2 = Room coefficient
- β_3 = Location coefficient

Step 3: Prediction

For a new apartment:

- Floor Area = 1100 sq.ft
- Rooms = 3
- Location = City

The model predicts the apartment rent.

Result:

The predicted rent is approximately

₹26,500 per month (*sample prediction*).

Conclusion:

Linear Regression helps estimate apartment rent accurately using important features like floor area, number of rooms, and location. This helps landlords and customers estimate a fair rental price.

Python Code:

```
1 import pandas as pd
2 from sklearn.linear_model import LinearRegression
3
4 # Sample dataset
5 data = {
6     "Area": [800, 1000, 1200, 1500, 900],
7     "Rooms": [2, 3, 3, 4, 2],
8     "Location": [1, 1, 0, 1, 0], # City=1, Suburb=0
9     "Rent": [18000, 25000, 22000, 32000, 19000]
10 }
11
12 df = pd.DataFrame(data)
13
14 X = df[["Area", "Rooms", "Location"]]
15 y = df["Rent"]
16
17 model = LinearRegression()
18 model.fit(X, y)
19
20 # Predict rent
21 new_house = [[1100, 3, 1]]
22 prediction = model.predict(new_house)
23
24 print("Predicted Rent = ₹", prediction[0])
```

Sample Input:

Floor Area = 1100 sq.ft

Rooms = 3

Location = City

Sample Output:

```
... Predicted Rent = ₹ 24999.999999999996
```

Question 2:

Scenario: Use a linear classification model to categorize emails as spam or non-spam using word frequency features in a mail service system.

Answer:**Introduction:**

Linear Classification is a supervised machine learning algorithm used to classify data into different categories. In this scenario, it is used to identify whether an email is Spam or Not Spam based on the frequency of words such as "Offer", "Free", and "Win".

Given Data (Sample):

Offer	Free	Win	Email Type
3	2	1	Spam
0	1	0	Not Spam
2	3	2	Spam
1	0	0	Not Spam
4	2	3	Spam

Solution:**Step 1: Collect Email Data**

Collect emails and count how many times spam-related words appear, such as:

- Offer
- Free

- Win

Step 2: Train the Model

The Linear Classification model learns the relationship between word frequencies and the email category.

General Equation:

$$y = w_0 + w_1(\text{Offer}) + w_2(\text{Free}) + w_3(\text{Win})$$

where,

- y = Predicted class
- w_0 = Bias
- w_1, w_2, w_3 = Weights of the features

Step 3: Prediction

For a new email:

- Offer = 2
- Free = 1
- Win = 2

The model predicts whether the email is Spam or Not Spam.

Result:

The email is classified as Spam because it contains several spam-related words.

Conclusion:

Linear Classification is simple and effective for email filtering. It helps mail services automatically detect spam emails and protect users from unwanted messages.

Python Code:

```
1 import pandas as pd
2 from sklearn.linear_model import LogisticRegression
3
4 # Sample dataset
5 data = {
6     "Offer": [3, 0, 2, 1, 4],
7     "Free": [2, 1, 3, 0, 2],
8     "Win": [1, 0, 2, 0, 3],
9     "Spam": [1, 0, 1, 0, 1]
10 }
11
12 df = pd.DataFrame(data)
13
14 X = df[["Offer", "Free", "Win"]]
15 y = df["Spam"]
16
17 model = LogisticRegression()
18 model.fit(X, y)
19
20 # Predict a new email
21 new_email = [[2, 1, 2]]
22 prediction = model.predict(new_email)
23
24 if prediction[0] == 1:
25     print("Spam Email")
26 else:
27     print("Not Spam")
```

Sample Input:

Offer = 2

Free = 1

Win = 2

Sample Output:

... Spam Email

Question 3:

Scenario: Apply the Naïve Bayes algorithm to predict whether a patient has flu based on symptoms such as fever, cough, and headache.

Answer:

Introduction:

Naïve Bayes is a supervised machine learning algorithm used for classification. It is based on Bayes' Theorem and predicts the class with the highest probability. In this scenario, it is used to predict whether a patient has Flu based on symptoms like Fever, Cough, and Headache.

Given Data (Sample):

Fever	Cough	Headache	Flu
1	1	1	Yes
1	1	0	Yes
0	1	1	No
1	0	1	Yes
0	0	0	No

Where:

- 1 = Symptom Present
- 0 = Symptom Absent

Solution:

Step 1: Collect Patient Data

Collect symptoms such as:

- Fever
- Cough
- Headache

along with whether the patient has flu.

Step 2: Train the Naïve Bayes Model

The model calculates the probability of each symptom for patients with and without flu using Bayes' Theorem.

Bayes' Formula:

$$P(A|B) = [P(B|A) \times P(A)] / P(B)$$

where,

- $P(A|B)$ = Posterior Probability
- $P(B|A)$ = Likelihood
- $P(A)$ = Prior Probability
- $P(B)$ = Evidence

Step 3: Prediction

For a new patient:

- Fever = 1
- Cough = 1
- Headache = 0

The model calculates the probability and predicts whether the patient has flu.

Result:

The patient is predicted to have Flu (Yes) because the symptoms closely match previous flu cases.

Conclusion:

Naïve Bayes is a fast and simple algorithm for medical diagnosis. It predicts diseases based on symptom probabilities and helps doctors make quick decisions.

Python Code:

```
1 import pandas as pd
2 from sklearn.naive_bayes import GaussianNB
3
4 # Sample dataset
5 data = {
6     "Fever": [1,1,0,1,0],
7     "Cough": [1,1,1,0,0],
8     "Headache": [1,0,1,1,0],
9     "Flu": [1,1,0,1,0]
10 }
11
12 df = pd.DataFrame(data)
13
14 X = df[["Fever", "Cough", "Headache"]]
15 y = df["Flu"]
16
17 model = GaussianNB()
18 model.fit(X,y)
19
20 # Predict a new patient
21 new_patient = [[1,1,0]]
22 prediction = model.predict(new_patient)
23
24 if prediction[0] == 1:
25     print("Flu: Yes")
26 else:
27     print("Flu: No")
```

Sample Input:

Fever = 1

Cough = 1

Headache = 0

Sample Output:

... Flu: Yes

Question 4:

Scenario: Use discriminant functions to classify loan applicants into eligible and non-eligible groups based on income and credit score.

Answer:

Introduction:

Discriminant Analysis is a supervised machine learning technique used for classification. It separates data into different groups using a discriminant function. In this scenario, it is used to classify loan applicants as Eligible or Not Eligible based on their Income and Credit Score.

Given Data:

Income (₹)	Credit Score	Loan Status
30000	650	Not Eligible
45000	700	Eligible
50000	750	Eligible
25000	600	Not Eligible
60000	800	Eligible

Solution:

Step 1: Collect Applicant Data

Collect details such as:

- Income
- Credit Score
- Loan Status

Step 2: Train the Model

The Discriminant Function learns how income and credit score affect loan approval.

General Equation:

$$g(x) = w_1(\text{Income}) + w_2(\text{Credit Score}) + b$$

where,

- $g(x)$ = Discriminant Function
- w_1, w_2 = Weights
- b = Bias

Step 3: Prediction

For a new applicant:

- Income = ₹48,000
- Credit Score = 720

The model calculates the discriminant score and predicts whether the applicant is eligible.

Result:

The applicant is classified as Eligible because the income and credit score are good.

Conclusion:

Discriminant Analysis helps banks classify loan applicants quickly. It reduces manual work and improves the accuracy of loan approval decisions.

Python Code:

```
1 import pandas as pd
2 from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
3
4 # Sample dataset
5 data = {
6     "Income": [30000, 45000, 50000, 25000, 60000],
7     "CreditScore": [650, 700, 750, 600, 800],
8     "Loan": [0, 1, 1, 0, 1]
9 }
10
11 df = pd.DataFrame(data)
12
13 X = df[["Income", "CreditScore"]]
14 y = df["Loan"]
15
16 model = LinearDiscriminantAnalysis()
17 model.fit(X, y)
18
19 # Predict new applicant
20 new_applicant = [[48000, 720]]
21 prediction = model.predict(new_applicant)
22
23 if prediction[0] == 1:
24     print("Eligible for Loan")
25 else:
26     print("Not Eligible")
```

Sample Input:

Income = ₹48,000

Credit Score = 720

Sample Output:

... Eligible for Loan

Question 5:

Scenario: Apply a probabilistic generative model to classify handwritten digits in an Optical Character Recognition (OCR) system.

Answer:

Introduction:

A Probabilistic Generative Model learns the probability distribution of each class and predicts the most likely class for new data. In an Optical Character Recognition (OCR) system, it is used to recognize handwritten digits (0–9). A commonly used probabilistic generative model is Naïve Bayes.

Given Data (Sample):

Pixel1	Pixel2	Pixel3	Digit
0	1	1	0
1	0	1	1
1	1	0	2
0	0	1	3
1	1	1	4

(Pixel values are sample features extracted from handwritten images.)

Solution:

Step 1: Collect Image Data

Collect images of handwritten digits (0–9).

Each image is converted into pixel values, which are used as input features.

Step 2: Train the Model

The Naïve Bayes classifier learns the probability of each digit based on the pixel values.

Using Bayes' Theorem:

$$P(A|B) = [P(B|A) \times P(A)] / P(B)$$

where,

- $P(A|B)$ = Posterior Probability
- $P(B|A)$ = Likelihood
- $P(A)$ = Prior Probability
- $P(B)$ = Evidence

Step 3: Prediction

For a new handwritten digit image:

- Pixel1 = 1
- Pixel2 = 0
- Pixel3 = 1

The model predicts the digit based on the highest probability.

Result:

The model predicts the handwritten digit as:

Digit = 1

Conclusion:

Probabilistic Generative Models such as Naïve Bayes can classify handwritten digits efficiently. They are simple, fast, and widely used in OCR applications for recognizing handwritten numbers.

Python Code:

```
1 import pandas as pd
2 from sklearn.naive_bayes import GaussianNB
3
4 # Sample dataset
5 data = {
6     "Pixel1": [0,1,1,0,1],
7     "Pixel2": [1,0,1,0,1],
8     "Pixel3": [1,1,0,1,1],
9     "Digit": [0,1,2,3,4]
10 }
11
12 df = pd.DataFrame(data)
13
14 X = df[["Pixel1","Pixel2","Pixel3"]]
15 y = df["Digit"]
16
17 model = GaussianNB()
18 model.fit(X,y)
19
20 # Predict a new digit
21 new_digit = [[1,0,1]]
22 prediction = model.predict(new_digit)
23
24 print("Predicted Digit:", prediction[0])
```

Sample Input:

Pixel1 = 1

Pixel2 = 0

Pixel3 = 1

Sample Output:

... Predicted Digit: 1

Question 6:

Scenario: Implement a probabilistic discriminative model to predict customer churn from service usage data in a telecom scenario.

Answer:

Introduction:

A Probabilistic Discriminative Model predicts the probability of a class directly from the input features. One of the most commonly used models is Logistic Regression. In this scenario, it is used to predict whether a telecom customer will leave the service (Churn) or continue using it based on service usage data.

Given Data (Sample):

Monthly Charges (₹)	Data Usage (GB)	Call Minutes	Churn
500	20	300	No
700	35	500	Yes
450	18	250	No
900	40	600	Yes
600	25	350	No

Where:

- Churn = Yes → Customer leaves the service.
- Churn = No → Customer continues the service.

Solution:

Step 1: Collect Customer Data

Collect customer information such as:

- Monthly Charges
- Data Usage

- Call Minutes
- Churn Status

Step 2: Train the Model

Logistic Regression learns the relationship between customer usage and churn.

The logistic regression equation is:

$$P(Y=1) = 1 / (1 + e^{-z})$$

where,

$$Z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3$$

- X_1 = Monthly Charges
- X_2 = Data Usage
- X_3 = Call Minutes

Step 3: Prediction

For a new customer:

- Monthly Charges = ₹650
- Data Usage = 30 GB
- Call Minutes = 400

The model predicts whether the customer will churn.

Result:

The customer is predicted as:

Churn = No

because the churn probability is less than 0.5.

Conclusion:

Logistic Regression is an effective probabilistic discriminative model for customer churn prediction. It helps telecom companies identify customers who are likely to leave and take preventive actions.

Python Code:

```
1 import pandas as pd
2 from sklearn.linear_model import LogisticRegression
3
4 # Sample dataset
5 data = {
6     "Charges": [500, 700, 450, 900, 600],
7     "DataUsage": [20, 35, 18, 40, 25],
8     "CallMinutes": [300, 500, 250, 600, 350],
9     "Churn": [0, 1, 0, 1, 0]
10 }
11
12 df = pd.DataFrame(data)
13
14 X = df[["Charges", "DataUsage", "CallMinutes"]]
15 y = df["Churn"]
16
17 model = LogisticRegression()
18 model.fit(X, y)
19
20 # Predict new customer
21 new_customer = [[650, 30, 400]]
22 prediction = model.predict(new_customer)
23
24 if prediction[0] == 1:
25     print("Customer will Churn")
26 else:
27     print("Customer will Not Churn")
```

Sample Input:

Monthly Charges = ₹650

Data Usage = 30 GB

Call Minutes = 400

Sample Output:

... Customer will Not Churn

Question 7:

Scenario: Apply Bayesian Logistic Regression to predict whether a customer will purchase a product after viewing online advertisements.

Answer:

Introduction:

Bayesian Logistic Regression is a supervised machine learning algorithm used for binary classification. It predicts the probability of an event occurring by combining logistic regression with Bayesian probability. In this scenario, it predicts whether a customer will purchase a product after viewing online advertisements.

Given Data (Sample):

Ads Viewed	Time Spent (min)	Age	Purchased
5	10	22	Yes
2	5	30	No
8	15	25	Yes
1	3	40	No
6	12	28	Yes

Where:

- Purchased = Yes → Customer buys the product.
- Purchased = No → Customer does not buy the product.

Solution:

Step 1: Collect Customer Data

Collect customer details such as:

- Number of Ads Viewed
- Time Spent on Advertisement
- Age
- Purchase Status

Step 2: Train the Model

The Bayesian Logistic Regression model learns the relationship between customer behavior and product purchase.

The logistic regression equation is:

$$P(Y=1) = 1 / (1 + e^{-z})$$

where,

$$Z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3$$

- X_1 = Ads Viewed
- X_2 = Time Spent
- X_3 = Age

Step 3: Prediction

For a new customer:

- Ads Viewed = 7
- Time Spent = 14 minutes
- Age = 26 years

The model predicts whether the customer will purchase the product.

Result:

The customer is predicted to:

Purchase the Product (Yes)

because the predicted probability is greater than 0.5.

Conclusion:

Bayesian Logistic Regression helps companies predict customer purchasing behaviour. It allows businesses to identify potential buyers and improve online advertising strategies.

Python Code:

```
1 import pandas as pd
2 from sklearn.linear_model import LogisticRegression
3
4 # Sample dataset
5 data = {
6     "AdViewed": [5, 2, 8, 1, 6],
7     "TimeSpent": [10, 5, 15, 3, 12],
8     "Age": [22, 30, 25, 40, 28],
9     "Purchased": [1, 0, 1, 0, 1]
10 }
11
12 df = pd.DataFrame(data)
13
14 X = df[["AdViewed", "TimeSpent", "Age"]]
15 y = df["Purchased"]
16
17 model = LogisticRegression()
18 model.fit(X, y)
19
20 # Predict new customer
21 new_customer = [[7, 14, 26]]
22 prediction = model.predict(new_customer)
23
24 if prediction[0] == 1:
25     print("Customer will Purchase the Product")
26 else:
27     print("Customer will Not Purchase the Product")
```

Sample Input:

Ads Viewed = 7

Time Spent = 14 minutes

Age = 26 years

Sample Output:

... Customer will Purchase the Product

Question 8:

Scenario: Use Linear Regression to forecast monthly sales for a retail company based on advertising expenditure and seasonal demand.

Answer:

Introduction:

Linear Regression is a supervised machine learning algorithm used to predict continuous values. In this scenario, it is used to forecast the monthly sales of a retail company based on advertising expenditure and seasonal demand.

Given Data (Sample):

Advertising (₹)	Seasonal Demand	Monthly Sales (₹)
10000	High	80000
15000	High	100000
8000	Low	60000
12000	Medium	85000
18000	High	120000

Note: Encode Seasonal Demand as:

- High = 2
- Medium = 1
- Low = 0

Solution:

Step 1: Collect Data

Collect:

- Advertising Expenditure
- Seasonal Demand
- Monthly Sales

Step 2: Train the Model

The Linear Regression model learns the relationship between advertising, seasonal demand, and sales.

General Equation:

$$\text{Sales} = \beta_0 + \beta_1(\text{Advertising}) + \beta_2(\text{Seasonal Demand})$$

where,

- β_0 = Intercept
- β_1 = Advertising coefficient
- β_2 = Seasonal demand coefficient

Step 3: Prediction

For a new month:

- Advertising = ₹14,000
- Seasonal Demand = High (2)

The model predicts the expected monthly sales.

Result:

The predicted monthly sales are approximately:

₹95,000

(Sample prediction)

Conclusion:

Linear Regression helps companies estimate future sales using advertising expenses and seasonal demand. This helps businesses improve marketing strategies and inventory planning.

Python Code:

```
1 import pandas as pd
2 from sklearn.linear_model import LinearRegression
3
4 # Sample dataset
5 data = {
6     "Advertising": [10000, 15000, 8000, 12000, 18000],
7     "Season": [2, 2, 0, 1, 2], # High=2, Medium=1, Low=0
8     "Sales": [80000, 100000, 60000, 85000, 120000]
9 }
10
11 df = pd.DataFrame(data)
12
13 X = df[["Advertising", "Season"]]
14 y = df["Sales"]
15
16 model = LinearRegression()
17 model.fit(X, y)
18
19 # Predict new month's sales
20 new_data = [[14000, 2]]
21 prediction = model.predict(new_data)
22
23 print("Predicted Monthly Sales = ₹", prediction[0])
```

Sample Input:

Advertising = ₹14,000

Seasonal Demand = High

Sample Output:

```
... Predicted Monthly Sales = ₹ 98474.57627118644
```


Question 9:

Scenario: Apply Naïve Bayes classification in a news portal to categorize articles into Sports, Politics, and Entertainment.

Answer:

Introduction:

Naïve Bayes is a supervised machine learning algorithm used for classification. It is based on Bayes' Theorem and predicts the class with the highest probability. In this scenario, it is used to classify news articles into Sports, Politics, or Entertainment based on the words present in the article.

Given Data (Sample):

Keywords	Category
Match, Goal, Team	Sports
Election, Government, Vote	Politics
Movie, Actor, Music	Entertainment
Cricket, Trophy, Player	Sports
Minister, Parliament, Policy	Politics

Solution:

Step 1: Collect News Articles

Collect news articles and identify important keywords such as:

- Match
- Goal
- Team
- Election
- Government
- Movie
- Actor

Step 2: Train the Model

The Naïve Bayes algorithm calculates the probability of each category based on the keywords.

Bayes' Formula

$$P(A|B) = [P(B|A) \times P(A)] / P(B)$$

where,

- $P(A|B)$ = Posterior Probability
- $P(B|A)$ = Likelihood
- $P(A)$ = Prior Probability
- $P(B)$ = Evidence

Step 3: Prediction

Suppose a new article contains the words:

- Match
- Team
- Trophy

The model calculates the probability for each category and predicts the one with the highest probability.

Result:

The article is classified as:

Sports

because most of its keywords belong to the Sports category.

Conclusion:

Naïve Bayes is a fast and efficient algorithm for text classification. It helps news websites automatically organize articles into different categories, making news searching and browsing easier.

Python Code:

```
1 from sklearn.feature_extraction.text import CountVectorizer
2 from sklearn.naive_bayes import MultinomialNB
3
4 # Sample dataset
5 texts = [
6     "match goal team",
7     "election government vote",
8     "movie actor music",
9     "cricket trophy player",
10    "minister parliament policy"
11 ]
12
13 labels = [
14     "Sports",
15     "Politics",
16     "Entertainment",
17     "Sports",
18     "Politics"
19 ]
20
21 # Convert text into numbers
22 vectorizer = CountVectorizer()
23 X = vectorizer.fit_transform(texts)
24
25 # Train model
26 model = MultinomialNB()
27 model.fit(X, labels)
28
29 # Predict new article
30 new_article = ["match trophy team"]
31 X_new = vectorizer.transform(new_article)
32
33 prediction = model.predict(X_new)
34
35 print("Category:", prediction[0])
```

Sample Input:

Article:

"match trophy team"

Sample Output:

... Category: Sports

Question 10:

Scenario: Use Bayesian Logistic Regression to predict student placement outcomes based on CGPA, coding skills, and aptitude scores.

Answer:

Introduction:

Bayesian Logistic Regression is a supervised machine learning algorithm used for binary classification. It predicts the probability of an event occurring. In this scenario, it is used to predict whether a student will be Placed or Not Placed based on CGPA, Coding Skills, and Aptitude Score.

Given Data (Sample):

CGPA	Coding Score	Aptitude Score	Placement
8.5	90	85	Placed
7.0	70	65	Not Placed
9.2	95	90	Placed
6.8	60	55	Not Placed
8.0	85	80	Placed

Where:

- Placed = 1
- Not Placed = 0

Solution:

Step 1: Collect Student Data

Collect the following details:

- CGPA
- Coding Skills Score
- Aptitude Score
- Placement Status

Step 2: Train the Model

The Bayesian Logistic Regression model learns the relationship between student performance and placement.

The logistic regression equation is:

$$P(Y=1) = 1 / (1 + e^{-z})$$

where,

$$Z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3$$

- X_1 = CGPA
- X_2 = Coding Score
- X_3 = Aptitude Score

Step 3: Prediction

For a new student:

- CGPA = 8.4
- Coding Score = 88
- Aptitude Score = 82

The model predicts whether the student will be placed.

Result:

The student is predicted to be:

Placed

because the predicted probability is greater than 0.5.

Conclusion:

Bayesian Logistic Regression helps colleges and companies predict placement outcomes using students' academic and technical performance. It provides quick and accurate predictions for placement analysis.

Python Code:

```
1 import pandas as pd
2 from sklearn.linear_model import LogisticRegression
3
4 # Sample dataset
5 data = {
6     "CGPA": [8.5, 7.0, 9.2, 6.8, 8.0],
7     "Coding": [90, 70, 95, 60, 85],
8     "Aptitude": [85, 65, 90, 55, 80],
9     "Placed": [1, 0, 1, 0, 1]
10 }
11
12 df = pd.DataFrame(data)
13
14 X = df[["CGPA", "Coding", "Aptitude"]]
15 y = df["Placed"]
16
17 model = LogisticRegression()
18 model.fit(X, y)
19
20 # Predict new student
21 new_student = [[8.4, 88, 82]]
22 prediction = model.predict(new_student)
23
24 if prediction[0] == 1:
25     print("Student is Placed")
26 else:
27     print("Student is Not Placed")
```

Sample Input:

CGPA = 8.4

Coding Score = 88

Aptitude Score = 82

Sample Output:

... Student is Placed